



Word (computer architecture)

In computing, a **word** is a fixed-sized datum handled as the natural or historical unit of data by the instruction set or the hardware of a processor. The number of bits or digits^[a] in a word (the *word size*, *word width*, or *word length*) is an important characteristic of any specific processor design or computer architecture.

The size of a word is reflected in many aspects of a computer's structure and operation; the majority of the registers in a processor are usually word-sized and the largest datum that can be transferred to and from the working memory in a single operation is a word in many (not all) architectures. The largest possible address size, used to designate a location in memory, is typically a hardware word (here, "hardware word" means the full-sized natural word of the processor, as opposed to any other definition used).

Several of the earliest computers (and a few modern as well) use binary-coded decimal rather than plain binary, typically having a word size of 10 or 12 decimal digits, and some early decimal computers have no fixed word length at all. Early binary systems tended to use word lengths that were some multiple of 6-bits, with the 36-bit word being especially common on mainframe computers. The introduction of ASCII led to the move to systems with word lengths that were a multiple of 8-bits, with 16-bit machines being popular in the 1970s before the move to modern processors with 32 or 64 bits.^[1] Special-purpose designs like digital signal processors, may have any word length from 4 to 80 bits.^[1]

The size of a word can sometimes differ from the expected due to backward compatibility with earlier computers. If multiple compatible variations or a family of processors share a common architecture and instruction set but differ in their word sizes, their documentation and software may become notationally complex to accommodate the difference (see Size families below).

Uses of words

Depending on how a computer is organized, word-size units may be used for:

Fixed-point numbers

Holders for fixed point, usually integer, numerical values may be available in one or in several different sizes, but one of the sizes available will almost always be the word. The other sizes, if any, are likely to be multiples or fractions of the word size. The smaller sizes are normally used only for efficient use of memory; when loaded into the processor, their values usually go into a larger holder, at least word sized.

Floating-point numbers

Holders for floating-point numerical values are typically either a word or a multiple of a word.

Addresses

Holders for memory addresses must be of a size capable of expressing the needed range of values but not be excessively large, so often the size used is the word though it can also be a multiple or fraction of the word size.

Registers

Processor registers are designed with a size appropriate for the type of data they hold, e.g. integers, floating-point numbers, or addresses. Many computer architectures use general-purpose registers that are capable of storing data in multiple representations.

Memory-processor transfer

When the processor reads from the memory subsystem into a register or writes a register's value to memory, the amount of data transferred is often a word. Historically, this amount of bits which could be transferred in one cycle was also called a *catena* in some environments (such as the Bull Gamma 60).^{[2][3]} In simple memory subsystems, the word is transferred over the memory data bus, which typically has a width of a word or half-word. In memory subsystems that use caches, the word-sized transfer is the one between the processor and the first level of cache; at lower levels of the memory hierarchy larger transfers (which are a multiple of the word size) are normally used.

Unit of address resolution

In a given architecture, successive address values almost^[b] always designate successive units of memory; this unit is the unit of address resolution. In most computers, the unit is either a character (e.g. a byte) or a word. (A few computers have used bit resolution.) If the unit is a word, then a larger amount of memory can be accessed using an address of a given size at the cost of added complexity to access individual characters. On the other hand, if the unit is a byte, then individual characters can be addressed (i.e. selected during the memory operation).

Instructions

Machine instructions are normally the size of the architecture's word or a fraction, such as in RISC architectures, or a multiple of the "char" size that is a fraction of it. This is a natural choice since instructions and data usually share the same memory subsystem. In Harvard architectures the word sizes of instructions and data need not be related, as instructions and data are stored in different memories; for example, the processor in the 1ESS electronic telephone switch has 37-bit instructions and 23-bit data words.

Word size choice

When a computer architecture is designed, the choice of a word size is of substantial importance. There are design considerations which encourage particular bit-group sizes for particular uses (e.g. for addresses), and these considerations point to different sizes for different uses. However, considerations of economy in design strongly push for one size, or a very few sizes related by multiples or fractions (submultiples) to a primary size. That preferred size becomes the word size of the architecture.

Character size was in the past (pre-variable-sized character encoding) one of the influences on unit of address resolution and the choice of word size. Before the mid-1960s, characters were most often stored in six bits; this allowed no more than 64 characters, so the alphabet was limited to upper case. Since it is efficient in time and space to have the word size be a multiple of the character size, word sizes in this period were usually multiples of 6 bits (in binary machines). A common choice then was the 36-bit word, which is also a good size for the numeric properties of a floating point format.

After the introduction of the IBM System/360 design, which uses eight-bit characters and supports lower-case letters, the standard size of a character (or more accurately, a byte) becomes eight bits. Word sizes thereafter are naturally multiples of eight bits, with 16, 32, and 64 bits being commonly used.

Variable-word architectures

Early machine designs included some that used what is often termed a *variable word length*. In this type of organization, an operand has no fixed length. Depending on the machine and the instruction, the length might be denoted by a count field, by a delimiting character, or by an additional bit called, e.g., flag, word mark. Such machines often use binary-coded decimal in 4-bit digits, or in 6-bit characters, for numbers. This class of machines includes the IBM 702, IBM 705, IBM 7080, IBM 7010, IBM 1400 series, IBM 1620, RCA 301, RCA 3301 and UNIVAC 1050.

Most of these machines work on one unit of memory at a time and since each instruction or datum is several units long, each instruction takes several cycles just to access memory. These machines are often quite slow because of this. For example, instruction fetches on an IBM 1620 Model I take 8 cycles (160 μ s) just to read the 12 digits of the instruction (the Model II reduced this to 6 cycles, or 4 cycles if the instruction did not need both address fields). Instruction execution takes a variable number of cycles, depending on the size of the operands.

Word, bit and byte addressing

The memory model of an architecture is strongly influenced by the word size. In particular, the resolution of a memory address, that is, the smallest unit that can be designated by an address, has often been chosen to be the word. In this approach, the word-addressable machine approach, address values which differ by one designate adjacent memory words. This is natural in machines which deal almost always in word (or multiple-word) units, and has the advantage of allowing instructions to use minimally sized fields to contain addresses, which can permit a smaller instruction size or a larger variety of instructions.

When byte processing is to be a significant part of the workload, it is usually more advantageous to use the byte, rather than the word, as the unit of address resolution. Address values which differ by one designate adjacent bytes in memory. This allows an arbitrary character within a character string to be addressed straightforwardly. A word can still be addressed, but the address to be used requires a few more bits than the word-resolution alternative. The word size needs to be an integer multiple of the character size in this organization. This addressing approach was used in the IBM 360, and has been the most common approach in machines designed since then.

When the workload involves processing fields of different sizes, it can be advantageous to address to the bit. Machines with bit addressing may have some instructions that use a programmer-defined byte size and other instructions that operate on fixed data sizes. As an example, on the IBM 7030^[4] ("Stretch"), a floating point instruction can only address words while an integer arithmetic instruction can specify a field length of 1-64 bits, a byte size of 1-8 bits and an accumulator offset of 0-127 bits.

In a byte-addressable machine with storage-to-storage (SS) instructions, there are typically move instructions to copy one or multiple bytes from one arbitrary location to another. In a byte-oriented (byte-addressable) machine without SS instructions, moving a single byte from one arbitrary location to another is typically:

1. LOAD the source byte
2. STORE the result back in the target byte

Individual bytes can be accessed on a word-oriented machine in one of two ways. Bytes can be manipulated by a combination of shift and mask operations in registers. Moving a single byte from one arbitrary location to another may require the equivalent of the following:

1. **LOAD** the word containing the source byte
2. **SHIFT** the source word to align the desired byte to the correct position in the target word
3. **AND** the source word with a mask to zero out all but the desired bits
4. **LOAD** the word containing the target byte
5. **AND** the target word with a mask to zero out the target byte
6. **OR** the registers containing the source and target words to insert the source byte
7. **STORE** the result back in the target location

Alternatively many word-oriented machines implement byte operations with instructions using special *byte pointers* in registers or memory. For example, the PDP-10 byte pointer contained the size of the byte in bits (allowing different-sized bytes to be accessed), the bit position of the byte within the word, and the word address of the data. Instructions could automatically adjust the pointer to the next byte on, for example, load and deposit (store) operations.

Powers of two

Different amounts of memory are used to store data values with different degrees of precision. The commonly used sizes are usually a power of two multiple of the unit of address resolution (byte or word). Converting the index of an item in an array into the memory address offset of the item then requires only a shift operation rather than a multiplication. In some cases this relationship can also avoid the use of division operations. As a result, most modern computer designs have word sizes (and other operand sizes) that are a power of two times the size of a byte.

Documentation for older computers with fixed word size commonly states memory sizes in words rather than bytes or characters. The documentation sometimes uses metric prefixes correctly, sometimes with rounding, e.g., 65 *kilowords* (kW) meaning for 65536 words, and sometimes uses them incorrectly, with *kilowords* (kW) meaning 1024 words (2^{10}) and megawords (MW) meaning 1,048,576 words (2^{20}). With standardization on 8-bit bytes and byte addressability, stating memory sizes in bytes, kilobytes, and megabytes with powers of 1024 rather than 1000 has become the norm, although there is some use of the IEC binary prefixes.

Size families

As computer designs have grown more complex, the central importance of a single word size to an architecture has decreased. Although more capable hardware can use a wider variety of sizes of data, market forces exert pressure to maintain backward compatibility while extending processor capability. As a result, what might have been the central word size in a fresh design has to coexist as an alternative size to the original word size in a backward compatible design. The original word size remains available in future designs, forming the basis of a size family.

In the mid-1970s, DEC designed the VAX to be a 32-bit successor of the 16-bit PDP-11. They used *word* for a 16-bit quantity, while *longword* referred to a 32-bit quantity; this terminology is the same as the terminology used for the PDP-11. This was in contrast to earlier machines, where the natural unit of

addressing memory would be called a *word*, while a quantity that is one half a word would be called a *halfword*. In fitting with this scheme, a VAX *quadword* is 64 bits. They continued this 16-bit word/32-bit longword/64-bit quadword terminology with the 64-bit Alpha.

Another example is the x86 family, of which processors of three different word lengths (16-bit, later 32- and 64-bit) have been released, while *word* continues to designate a 16-bit quantity. As software is routinely ported from one word-length to the next, some APIs and documentation define or refer to an older (and thus shorter) word-length than the full word length on the CPU that software may be compiled for. Also, similar to how bytes are used for small numbers in many programs, a shorter word (16 or 32 bits) may be used in contexts where the range of a wider word is not needed (especially where this can save considerable stack space or cache memory space). For example, Microsoft's Windows API maintains the programming language definition of *WORD* as 16 bits, despite the fact that the API may be used on a 32- or 64-bit x86 processor, where the standard word size would be 32 or 64 bits, respectively. Data structures containing such different sized words refer to them as:

- *WORD* (16 bits/2 bytes)
- *DWORD* (32 bits/4 bytes)
- *QWORD* (64 bits/8 bytes)

A similar phenomenon has developed in Intel's x86 assembly language – because of the support for various sizes (and backward compatibility) in the instruction set, some instruction mnemonics carry "d" or "q" identifiers denoting "double-", "quad-" or "double-quad-", which are in terms of the architecture's original 16-bit word size.

An example with a different word size is the IBM System/360 family. In the System/360 architecture, System/370 architecture and System/390 architecture, there are 8-bit *bytes*, 16-bit *halfwords*, 32-bit *words* and 64-bit *doublewords*. The z/Architecture, which is the 64-bit member of that architecture family, continues to refer to 16-bit *halfwords*, 32-bit *words*, and 64-bit *doublewords*, and additionally features 128-bit *quadwords*.

In general, new processors must use the same data word lengths and virtual address widths as does an older processor to have binary compatibility with that older processor.

Often carefully written source code – written with source-code compatibility and software portability in mind – can be recompiled to run on a variety of processors, even ones with different data word lengths or different address widths or both.

Table of word sizes

key: bit: <u>bits</u> , c: <u>characters</u> , d: <u>decimal digits</u> , w: <u>word size of architecture</u> , n: <u>variable size</u> , wm: <u>word mark</u> ^{[5][6]}							
Year	Computer architecture	Word size w	Integer sizes	Floating point sizes	Instruction sizes	Unit of address resolution	Char size
1837	<u>Babbage Analytical engine</u>	50 d	w	—	Five different cards were used for different functions, exact size of cards not known.	w	—
1941	<u>Zuse Z3</u>	22 bit	—	w	8 bit	w	—
1942	<u>ABC</u>	50 bit	w	—	—	—	—
1944	<u>Harvard Mark I</u>	23 d	w	—	24 bit	—	—
1946 (1948) {1953}	ENIAC (w/Panel #16 ^[7]) {w/Panel #26 ^[8] }	10 d	$w, 2w$ (w) { w }	—	— (2 d, 4 d, 6 d, 8 d) {2 d, 4 d, 6 d, 8 d}	— — { w }	—
1948	<u>Manchester Baby</u>	32 bit	w	—	w	w	—
1951	<u>UNIVAC I</u>	12 d	w	—	$\frac{1}{2}w$	w	1 d
1952	<u>IAS machine</u>	40 bit	w	—	$\frac{1}{2}w$	w	5 bit
1952	<u>Fast Universal Digital Computer M-2</u>	34 bit	$w?$	w	34 bit = 4-bit opcode plus 3×10 bit address	10 bit	—
1952	<u>IBM 701</u>	36 bit	$\frac{1}{2}w, w$	—	$\frac{1}{2}w$	$\frac{1}{2}w, w$	6 bit
1952	<u>UNIVAC 60</u>	n d	1 d, ... 10 d	—	—	—	2 d, 3 d
1952	<u>ARRA I</u>	30 bit	w	—	w	w	5 bit
1953	<u>IBM 702</u>	n c	0 c, ... 511 c	—	5 c	c	6 bit
1953	<u>UNIVAC 120</u>	n d	1 d, ... 10 d	—	—	—	2 d, 3 d
1953	<u>ARRA II</u>	30 bit	w	$2w$	$\frac{1}{2}w$	w	5 bit
1954 (1955)	IBM 650 (w/IBM 653)	10 d	w	— (w)	w	w	2 d
1954	IBM 704/IBM 709/IBM 7090	36 bit	w	w	w	w	6 bit
1954	<u>IBM 705</u>	n c	0 c, ... 255 c	—	5 c	c	6 bit
1954	<u>IBM NORC</u>	16 d	w	$w, 2w$	w	w	—
1956	<u>IBM 305</u>	n d	1 d, ... 100 d	—	10 d	d	1 d

1956	<u>ARMAC</u>	34 bit	w	w	$\frac{1}{2}w$	w	5 bit, 6 bit
1956	<u>LGP-30</u>	31 bit	w	—	16 bit	w	6 bit
1958	<u>UNIVAC II</u>	12 d	w	—	$\frac{1}{2}w$	w	1 d
1958	<u>SAGE</u>	32 bit	$\frac{1}{2}w$	—	w	w	6 bit
1958	<u>Autonetics Recomp II</u>	40 bit	w , 79 bit, 8 d, 15 d	$2w$	$\frac{1}{2}w$	$\frac{1}{2}w$, w	5 bit
1958	<u>ZEBRA</u>	33 bit	w , 65 bit	$2w$	w	w	5 bit
1958	<u>Setun</u>	6 trit (~9.5 bits) ^[c]	up to 6 tryte		up to 3 trytes		4 trit [?]
1958	<u>Electrologica X1</u>	27 bit	w	$2w$	w	w	5 bit, 6 bit
1959	<u>IBM 1401</u>	n c	1 c, ...	—	1 c, 2 c, 4 c, 5 c, 7 c, 8 c	c	6 bit + wm
1959 (TBD)	<u>IBM 1620</u>	n d	2 d, ...	— (4 d, ... 102 d)	12 d	d	2 d
1960	<u>LARC</u>	12 d	w , $2w$	w , $2w$	w	w	2 d
1960	<u>CDC 1604</u>	48 bit	w	w	$\frac{1}{2}w$	w	6 bit
1960	<u>IBM 1410</u>	n c	1 c, ...	—	1 c, 2 c, 6 c, 7 c, 11 c, 12 c	c	6 bit + wm
1960	<u>IBM 7070</u>	10 d ^[d]	w , 1-9 d	w	w	w , d	2 d
1960	<u>PDP-1</u>	18 bit	w	—	w	w	6 bit
1960	<u>Elliott 803</u>	39 bit					
1961	<u>IBM 7030 (Stretch)</u>	64 bit	1 bit, ... 64 bit, 1 d, ... 16 d	w	$\frac{1}{2}w$, w	bit (integer), $\frac{1}{2}w$ (branch), w (float)	1 bit, ... 8 bit
1961	<u>IBM 7080</u>	n c	0 c, ... 255 c	—	5 c	c	6 bit
1962	<u>GE-6xx</u>	36 bit	w , $2w$	w , $2w$, 80 bit	w	w	6 bit, 9 bit
1962	<u>UNIVAC III</u>	25 bit	w , $2w$, $3w$, $4w$, 6 d, 12 d	—	w	w	6 bit
1962	<u>Autonetics D- 17B Minuteman I Guidance Computer</u>	27 bit	11 bit, 24 bit	—	24 bit	w	—
1962	<u>UNIVAC 1107</u>	36 bit	$\frac{1}{6}w$, $\frac{1}{3}w$, $\frac{1}{2}w$, w	w	w	w	6 bit
1962	<u>IBM 7010</u>	n c	1 c, ...	—	1 c, 2 c, 6 c, 7 c, 11 c, 12 c	c	6 b + wm
1962	<u>IBM 7094/IBM 7040/IBM 7044</u>	36 bit	w	w , $2w$	w	w	6 bit

1962	<u>SDS 9 Series</u>	24 bit	w	$2w$	w	w	
1963 (1966)	<u>Apollo Guidance Computer</u>	15 bit	w	—	$w, 2w$	w	—
1963	<u>Saturn Launch Vehicle Digital Computer</u>	26 bit	w	—	13 bit	w	—
1964/ 1966	<u>PDP-6/PDP-10</u>	36 bit	w	$w, 2w$	w	w	6 bit 7 bit (typical) 9 bit
1964	<u>Titan</u>	48 bit	w	w	w	w	w
1964	<u>CDC 6600</u>	60 bit	w	w	$\frac{1}{4}w, \frac{1}{2}w$	w	6 bit
1964	<u>Autonetics D- 37C Minuteman II Guidance Computer</u>	27 bit	11 bit, 24 bit	—	24 bit	w	4 bit, 5 bit
1965	<u>Gemini Guidance Computer</u>	39 bit	26 bit	—	13 bit	13 bit, 26	—bit
1965	<u>IBM 1130</u>	16 bit	$w, 2w$	$2w, 3w$	$w, 2w$	w	8 bit
1965	<u>IBM System/360</u>	32 bit	$\frac{1}{2}w, w,$ 1 d, ... 16 d	$w, 2w$	$\frac{1}{2}w, w, 1\frac{1}{2}w$	8 bit	8 bit
1965	<u>UNIVAC 1108 and successors</u>	36 bit	$\frac{1}{6}w,$ $\frac{1}{4}w,$ $\frac{1}{3}w,$ $\frac{1}{2}w, w,$ $2w$	$w, 2w$	w	w	6 bit, 9 bit
1965	<u>PDP-8</u>	12 bit	w	—	w	w	8 bit
1965	<u>Electrologica X8</u>	27 bit	w	$2w$	w	w	6 bit, 7 bit
1966	<u>SDS Sigma 7</u>	32 bit	$\frac{1}{2}w, w$	$w, 2w$	w	8 bit	8 bit
1969	<u>Data General NOVA</u>	16 bit	w	$2w, 4w$	w	w	w
1969	<u>Four-Phase Systems AL1</u>	8 bit	w	—	6 bits	no address	?
1970	<u>MP944</u>	20 bit	w	—	?	?	?
1970	<u>PDP-11</u>	16 bit	w	$2w, 4w$	$w, 2w, 3w$	8 bit	8 bit
1971	<u>Four-Phase System IV/70 (uses AL1 successor)</u>	24 bit	w	$2w$	w	w	8 bit
1971	<u>CDC STAR- 100</u>	64 bit	$\frac{1}{2}w, w$	$\frac{1}{2}w, w$	$\frac{1}{2}w, w$	bit	8 bit
1971	<u>TMS1802NC</u>	4 bit	w	—	?	?	—
1971	<u>Intel 4004</u>	4 bit	w, d	—	$2w, 4w$	w	—
1972	<u>Intel 8008</u>	8 bit	w	—	$w, 2w, 3w$	w	8 bit

1972	<u>Calcomp 900</u>	9 bit	w	—	$w, 2w$	w	8 bit
1973	<u>IMP-16</u>	16 bit	w	—	$w, 2w^{[e]}$	w	8 bit ^[f]
1974	<u>Intel 8080</u>	8 bit	$w, 2w, 2\text{ d}$	—	$w, 2w, 3w$	w	8 bit
1975	<u>ILLIAC IV</u>	64 bit	w	$w, \frac{1}{2}w$	w	w	—
1975	<u>Motorola 6800</u>	8 bit	$w, 2\text{ d}$	—	$w, 2w, 3w$	w	8 bit
1975	<u>MOS Technology 6501/6502</u>	8 bit	$w, 2\text{ d}$	—	$w, 2w, 3w$	w	8 bit
1976	<u>Cray-1</u>	64 bit	24 bit, w	w	$\frac{1}{4}w, \frac{1}{2}w$	w	8 bit
1976	<u>Zilog Z80</u>	8 bit	$w, 2w, 2\text{ d}$	—	$w, 2w, 3w, 4w$	w	8 bit
1976	<u>Signetics 8X300</u>	8 bit	w	—	16 bit	w	1-8 bit
1978 (1980)	16-bit x86 (<u>Intel 8086</u>) (w/floating point: <u>Intel 8087</u>)	16 bit	$\frac{1}{2}w, w, 2\text{ d}$	— ($2w, 4w, 5w, 17\text{ d}$)	$\frac{1}{2}w, w, \dots 7w$	8 bit	8 bit
1978	<u>VAX</u>	32 bit	$\frac{1}{4}w, \frac{1}{2}w, w, 1\text{ d}, \dots 31\text{ d}, 1\text{ bit}, \dots 32\text{ bit}$	$w, 2w$	$\frac{1}{4}w, \dots 14\frac{1}{4}w$	8 bit	8 bit
1979 (1984)	<u>Motorola 68000 series</u> (w/floating point)	32 bit	$\frac{1}{4}w, \frac{1}{2}w, w, 2\text{ d}$	— ($w, 2w, 2\frac{1}{2}w$)	$\frac{1}{2}w, w, \dots 7\frac{1}{2}w$	8 bit	8 bit
1981	<u>Intel iAPX 432</u>	32 bit	$\frac{1}{4}w, \frac{1}{2}w, w$	$w, 2w, 2\frac{1}{2}w$	6-321 bits	8 bit	8 bit
1985	<u>IA-32 (Intel 80386)</u> (w/floating point)	32 bit	$\frac{1}{4}w, \frac{1}{2}w, w$	— ($w, 2w, 80\text{ bit}$)	8 bit, $\dots 120\text{ bit}$ $\frac{1}{4}w \dots 3\frac{3}{4}w$	8 bit	8 bit
1985	<u>ARMv1</u>	32 bit	$\frac{1}{4}w, w$	—	w	8 bit	8 bit
1985	<u>MIPS I</u>	32 bit	$\frac{1}{4}w, \frac{1}{2}w, w$	$w, 2w$	w	8 bit	8 bit
1991	<u>Cray C90</u>	64 bit	32 bit, w	w	$\frac{1}{4}w, \frac{1}{2}w, 48\text{ bit}$	w	8 bit
1992	<u>Alpha</u>	64 bit	8 bit, $\frac{1}{4}w, \frac{1}{2}w, w$	$\frac{1}{2}w, w$	$\frac{1}{2}w$	8 bit	8 bit
1992	<u>PowerPC</u>	32 bit	$\frac{1}{4}w, \frac{1}{2}w, w$	$w, 2w$	w	8 bit	8 bit
1996	<u>ARMv4 (w/Thumb)</u>	32 bit	$\frac{1}{4}w, \frac{1}{2}w, w$	—	w ($\frac{1}{2}w, w$)	8 bit	8 bit
2000	<u>IBM z/Architecture</u>	64 bit ^[g]	8 bit, $\frac{1}{4}w, \frac{1}{2}w, w$	$\frac{1}{2}w, w, 2w$	$\frac{1}{4}w, \frac{1}{2}w, \frac{3}{4}w$	8 bit	8 bit, <u>UTF-16</u> , <u>UTF-32</u>

			1 d, ... 31 d				
2001	<u>IA-64</u>	64 bit	8 bit, $\frac{1}{4}w$, $\frac{1}{2}w$, w	$\frac{1}{2}w$, w	41 bit (in 128-bit bundles) ^[9]	8 bit	8 bit
2001	<u>ARMv6</u> (w/VFP)	32 bit	8 bit, $\frac{1}{2}w$, w	— (w , $2w$)	$\frac{1}{2}w$, w	8 bit	8 bit
2003	<u>x86-64</u>	64 bit	8 bit, $\frac{1}{4}w$, $\frac{1}{2}w$, w	$\frac{1}{2}w$, w , 80 bit	8 bit, ... 120 bit	8 bit	8 bit
2013	<u>AArch64</u>	64 bit	8 bit, $\frac{1}{4}w$, $\frac{1}{2}w$, w	$\frac{1}{2}w$, w	$\frac{1}{2}w$	8 bit	8 bit
Year	Computer architecture	Word size w	Integer sizes	Floating point sizes	Instruction sizes	Unit of address resolution	Char size
key: bit: <u>bits</u> , c: characters, d: <u>decimal digits</u> , w: word size of architecture, n: variable size, wm: <u>word mark</u>							

See also

- Syllable – Platform-specific data size used for some historical digital hardware

Notes

- Many early computers were decimal, and a few were ternary
- The UNIVAC 1005 addresses core using 5-bit Gray codes for row and column.
- The bit equivalent is computed by taking the amount of information entropy provided by the trit, which is $\log_2(3)$. This gives an equivalent of about 9.51 bits for 6 trits.
- Three-state sign
- IMP-16 supports two word instructions only if Extended Instruction Set CROM II is installed.
- Bytes not supported by IMP-16 Basic Instruction Set. Bytes only supported if Extended Instruction Set CROM II is installed. Even with CROM II, byte-sized memory writes are not supported.
- Although z/Architecture is an inherently 64-bit architecture, the term word still refers to a 32-bit quantity. However, in this table the sizes are treated as if a word were 64 bits.

References

- Beebe, Nelson H. F. (2017-08-22). "Chapter I. Integer arithmetic". *The Mathematical-Function Computation Handbook - Programming Using the MathCW Portable Software Library* (1 ed.). Salt Lake City, UT, US: Springer International Publishing AG. p. 970. doi:10.1007/978-3-319-64110-2 (https://doi.org/10.1007%2F978-3-319-64110-2). ISBN 978-3-319-64109-6. LCCN 2017947446 (https://lccn.loc.gov/2017947446). S2CID 30244721 (https://api.semanticscholar.org/CorpusID:30244721).
- Dreyfus, Phillippe (1958-05-08) [1958-05-06]. Written at Los Angeles, California, US. *System design of the Gamma 60* (https://www.computer.org/csdl/proceedings/afips/1958/5052/00/50520130.pdf) (PDF). Western Joint Computer Conference: Contrasts in Computers.

ACM, New York, NY, US. pp. 130–133. IRE-ACM-AIEE '58 (Western). Archived (<https://web.archive.org/web/20170403224547/https://www.computer.org/csdl/proceedings/afips/1958/5052/00/50520130.pdf>) (PDF) from the original on 2017-04-03. Retrieved 2017-04-03. "[...] Internal data code is used: Quantitative (numerical) data are coded in a 4-bit decimal code; qualitative (alpha-numerical) data are coded in a 6-bit alphanumerical code. The internal instruction code means that the instructions are coded in straight binary code.

As to the internal information length, the information quantum is called a "catena," and it is composed of 24 bits representing either 6 decimal digits, or 4 alphanumerical characters. This quantum must contain a multiple of 4 and 6 bits to represent a whole number of decimal or alphanumerical characters. Twenty-four bits was found to be a good compromise between the minimum 12 bits, which would lead to a too-low transfer flow from a parallel readout core memory, and 36 bits or more, which was judged as too large an information quantum. The catena is to be considered as the equivalent of a character in variable word length machines, but it cannot be called so, as it may contain several characters. It is transferred in series to and from the main memory.

Not wanting to call a "quantum" a word, or a set of characters a letter, (a word is a word, and a quantum is something else), a new word was made, and it was called a "catena." It is an English word and exists in Webster's although it does not in French. Webster's definition of the word catena is, "a connected series;" therefore, a 24-bit information item. The word catena will be used hereafter.

The internal code, therefore, has been defined. Now what are the external data codes? These depend primarily upon the information handling device involved. The Gamma 60 is designed to handle information relevant to any binary coded structure. Thus an 80-column punched card is considered as a 960-bit information item; 12 rows multiplied by 80 columns equals 960 possible punches; is stored as an exact image in 960 magnetic cores of the main memory with 2 card columns occupying one catena. [...]"

3. Blaauw, Gerrit Anne; Brooks, Jr., Frederick Phillips; Buchholz, Werner (1962). "4: Natural Data Units" (http://archive.computerhistory.org/resources/text/IBM/Stretch/pdfs/Buchholz_102636426.pdf) (PDF). In Buchholz, Werner (ed.). *Planning a Computer System – Project Stretch*. McGraw-Hill Book Company, Inc. / The Maple Press Company, York, PA. pp. 39–40. LCCN 61-10466 (<https://lccn.loc.gov/61-10466>). Archived (https://web.archive.org/web/20170403014651/http://archive.computerhistory.org/resources/text/IBM/Stretch/pdfs/Buchholz_102636426.pdf) (PDF) from the original on 2017-04-03. Retrieved 2017-04-03. "[...] Terms used here to describe the structure imposed by the machine design, in addition to bit, are listed below.

Byte denotes a group of bits used to encode a character, or the number of bits transmitted in parallel to and from input-output units. A term other than character is used here because a given character may be represented in different applications by more than one code, and different codes may use different numbers of bits (i.e., different byte sizes). In input-output transmission the grouping of bits may be completely arbitrary and have no relation to actual characters. (The term is coined from bite, but respelled to avoid accidental mutation to bit.) A word consists of the number of data bits transmitted in parallel from or to memory in one memory cycle. Word size is thus defined as a structural property of the memory. (The term catena was coined for this purpose by the designers of the Bull GAMMA 60 computer.) Block refers to the number of words transmitted to or from an input-output unit in response to a single input-output instruction. Block size is a structural property of an input-output unit; it may have been fixed by the design or left to be varied by the program. [...]"

4. "Format" (http://bitsavers.org/pdf/ibm/7030/22-6530-2_7030RefMan.pdf#page=59) (PDF). *Reference Manual 7030 Data Processing System* (http://bitsavers.org/pdf/ibm/7030/22-6530-2_7030RefMan.pdf) (PDF). IBM. August 1961. pp. 50–57. Retrieved 2021-12-15.
5. Blaauw, Gerrit Anne; Brooks, Jr., Frederick Phillips (1997). *Computer Architecture: Concepts and Evolution* (1 ed.). Addison-Wesley. ISBN 0-201-10557-8. (1213 pages) (NB. This is a single-volume edition. This work was also available in a two-volume version.)

6. Ralston, Anthony; Reilly, Edwin D. (1993). *Encyclopedia of Computer Science* (3rd ed.). Van Nostrand Reinhold. ISBN 0-442-27679-6.
7. Clippinger, Richard F. [in German] (1948-09-29). "A Logical Coding System Applied to the ENIAC (Electronic Numerical Integrator and Computer)" (<http://ftp.arl.mil/~mike/comphist/48eniac-coding/>). Aberdeen Proving Ground, Maryland, US: Ballistic Research Laboratories. Report No. 673; Project No. TB3-0007 of the Research and Development Division, Ordnance Department. Retrieved 2017-04-05.
8. Clippinger, Richard F. [in German] (1948-09-29). "A Logical Coding System Applied to the ENIAC" (<http://ftp.arl.mil/~mike/comphist/48eniac-coding/sec8.html>). Aberdeen Proving Ground, Maryland, US: Ballistic Research Laboratories. Section VIII: Modified ENIAC. Retrieved 2017-04-05.
9. "4. Instruction Formats" (<https://www.intel.com/content/dam/www/public/us/en/documents/manuals/itanium-architecture-vol-3-manual.pdf#page=302>) (PDF). *Intel Itanium Architecture Software Developer's Manual*. Vol. 3: Intel Itanium Instruction Set Reference. p. 3:293. Retrieved 2022-04-25. "Three instructions are grouped together into 128-bit sized and aligned containers called **bundles**. Each bundle contains three 41-bit **instruction slots** and a 5-bit template field."

Retrieved from "[https://en.wikipedia.org/w/index.php?title=Word_\(computer_architecture\)&oldid=1332363045](https://en.wikipedia.org/w/index.php?title=Word_(computer_architecture)&oldid=1332363045)"